Большая языковая модель с нуля

Учебное пособие для начинающих
по материалам книги Себастьяна Рашки
«Build a Large Language Model (From Scratch)»
и репозитория github.com/rasbt/LLMs-from-scratch

Продолжение серии: как устроена сама LLM изнутри.
После каждой главы — 5 вопросов для самопроверки.

Как читать это пособие

Это второе пособие серии. Первое («Рассуждающие языковые модели с нуля») отвечало на вопрос «как научить готовую LLM рассуждать». Это пособие отвечает на вопрос уровнем глубже: **как вообще устроена и создаётся сама LLM** — та самая «базовая модель», которую мы в первом пособии брали готовой и не заглядывали внутрь.

Книга Себастьяна Рашки «Build a Large Language Model (From Scratch)» строит GPT-подобную модель с абсолютного нуля на Python и PyTorch: от нарезки текста на токены до работающего чат-помощника. Мы пройдём тот же маршрут без кода — на уровне идей и механики, с небольшими псевдокод-вставками там, где они помогают (вы ведь разработчик).

Маршрут из 7 глав книги (плюс наша глава 0):

- **Глава 0** — повторение из первого пособия + математический минимум (векторы, обучение градиентом).
- **Глава 1** — общая картина: что такое LLM, трансформер и три этапа создания модели.
- **Глава 2** — данные: токенизация, BPE, эмбеддинги, подготовка обучающих пар.
- **Глава 3** — **внимание (attention)** — двигатель LLM и самая важная глава книги.
- **Глава 4** — собираем архитектуру GPT целиком из готовых деталей.
- **Глава 5** — предобучение: как модель учится и как измерять прогресс.
- **Глава 6** — дообучение №1: превращаем LLM в классификатор (спам-фильтр).
- **Глава 7** — дообучение №2: учим модель следовать инструкциям (путь к ChatGPT).

Правила прежние: читайте по порядку, после каждой главы отвечайте на 5 вопросов своими словами и присылайте в чат: «Глава N: 1) … 2) … 3) … 4) … 5) …». Я проверяю, разбираем ошибки, идём дальше.

Цветовые обозначения те же: жёлтый — «простыми словами», зелёный — аналогия, синий — термин, фиолетовый — ключевая мысль главы, красный — вопросы.

Глава 0. Повторение и математический минимум

Из первого пособия вы уже знаете главное, быстро освежим и добавим три недостающих кирпичика из математики — без них глава 3 (внимание) не поедет. Формул почти не будет, только смысл.

0.1 Что вы уже знаете (повторение за 60 секунд)

- LLM — предсказатель следующего токена; связный текст — результат цикла «предсказали → приклеили → повторили».
- Токен — кусок текста с номером; токенизатор кодирует текст в номера и декодирует обратно.
- На каждом шаге модель выдаёт логиты для всех токенов словаря → их превращают в вероятности → выбирают один (жадно или сэмплированием с температурой/top-p).
- Обучение = подбор весов на примерах; инференс = использование готовых весов; базовая модель умеет только продолжать текст.

В этой книге мы наконец увидим, **что происходит между** «подали токены» и «получили логиты» — внутри самой модели.

0.2 Кирпичик 1: вектор — это список чисел со смыслом

> **Вектор** — упорядоченный список чисел, например [0.2, −1.7, 3.1]. Количество чисел называется **размерностью**. В LLM всё — векторы: каждый токен внутри модели представлен вектором из сотен чисел (в GPT-2 — 768).

Зачем? Числа в векторе можно понимать как координаты точки в пространстве. А у точек есть расстояния и направления — значит, появляется геометрия смысла: векторы слов «кошка» и «собака» окажутся рядом, «кошка» и «карбюратор» — далеко. Смысл превращается в геометрию, а с геометрией компьютер работать умеет.

> **Скалярное произведение (dot product)** — операция над двумя векторами: перемножаем числа попарно и складываем: dot([1,2,3], [4,5,6]) = 1·4 + 2·5 + 3·6 = 32. Запомните его смысл: это **мера похожести/согласованности** двух векторов. Смотрят в одну сторону — число большое, перпендикулярны — около нуля, в противоположные — отрицательное. В главе 3 на этой операции будет построено всё внимание.

Матрица — таблица чисел (стопка векторов). **Умножение на матрицу** — способ превратить один вектор в другой (возможно, другой размерности). Все «знания» модели хранятся именно в матрицах, а работа модели — длинная цепочка умножений векторов на матрицы.

0.3 Кирпичик 2: нейросеть и её обучение

Нейросеть — это функция: вход (числа) → выход (числа), собранная из слоёв «умножить на матрицу и применить простую нелинейную функцию». Числа в матрицах — те самые **веса** (параметры).

Обучение устроено так:

1. Прогоняем пример через сеть, получаем предсказание.
2. Считаем **функцию потерь (loss)** — одно число «насколько предсказание плохое».
3. Вычисляем для каждого веса, в какую сторону его подкрутить, чтобы потеря уменьшилась. Этот расчёт называется **обратным распространением ошибки (backpropagation)**, а направление — **градиентом**.
4. Сдвигаем все веса на маленький шаг в нужную сторону (размер шага — **learning rate**). Это **градиентный спуск**.
5. Повторяем миллионы раз.

Аналогия. Настройка душа вслепую: крутим кран (вес), чувствуем, стало теплее или холоднее (loss), крутим дальше в удачную сторону маленькими шажками. Только кранов — миллиард, а backpropagation — математический способ узнать для всех кранов сразу, куда крутить.

0.4 Кирпичик 3: переобучение

Переобучение (overfitting) — ситуация, когда модель выучила обучающие примеры наизусть, а не закономерность, и на новых данных работает плохо. Чтобы его замечать, данные делят на **обучающую** выборку (на ней учимся) и **валидационную** (на ней только проверяем). Если потеря на обучающей падает, а на валидационной растёт — модель зубрит. Мы встретим это в главах 5–7.

Ключевая мысль главы 0. Внутри LLM всё — векторы и матрицы: токены становятся векторами, смысл — геометрией, похожесть измеряется скалярным произведением. Обучение — это цикл «предсказал → посчитал потерю → градиентом узнал, куда крутить веса → сдвинул на шажок», а честность проверяется на валидационной выборке.

Вопросы к главе 0

1. Зачем токены внутри модели представляют векторами? Что даёт «геометрия смысла»?
2. Что такое скалярное произведение и какой у него смысл применительно к двум векторам?
3. Опишите цикл обучения нейросети: какие шаги повторяются и какую роль играет функция потерь?
4. Что такое градиент и learning rate в аналогии с настройкой душа?
5. Что такое переобучение и как валидационная выборка помогает его обнаружить?

Глава 1. Общая картина: что мы будем строить

Первая глава книги — без кода: она объясняет, что такое LLM, откуда взялась архитектура «трансформер» и из каких этапов состоит создание модели. План всей книги — на одной схеме, запомните её:

```
ЭТАП 1: подготовка данных и архитектура     (главы 2-4)
ЭТАП 2: предобучение → базовая модель        (глава 5)
ЭТАП 3: дообучение → полезная модель         (главы 6-7)
```

1.1 Что такое LLM и почему она «большая»

LLM — нейросеть для работы с текстом, обученная предсказывать следующий токен. «Большая» — из-за количества параметров: от сотен миллионов (наш GPT-2 — 124 миллиона) до сотен миллиардов. Поразительный факт, мотивирующий всю область: из такой примитивной задачи («угадай следующее слово») при достаточном масштабе данных и параметров **сами собой** вырастают сложные умения — перевод, ответы на вопросы, написание кода. Их никто не программировал отдельно.

> **Простыми словами:** чтобы хорошо продолжать любой текст в интернете, приходится «понять» очень многое: грамматику, факты, логику, стили. Предсказание следующего слова — не цель, а педагогический трюк, заставляющий модель выучить язык и мир.

1.2 Трансформер и его ветви

> **Трансформер (transformer)** — архитектура нейросети, представленная в 2017 году в статье «Attention Is All You Need». Её сердце — **механизм внимания**, позволяющий каждому слову «смотреть» на все остальные слова текста и решать, какие из них важны. Почти все современные LLM — трансформеры.

Исходный трансформер создавался для перевода и состоял из двух половин: **энкодер** читает исходное предложение целиком, **декодер** порождает перевод слово за словом. Дальше семейство разделилось:

- ветка BERT (энкодеры) — модели для «понимания» текста: классификация, поиск;
- ветка **GPT (декодеры)** — модели для **генерации** текста. GPT расшифровывается как Generative Pre-trained Transformer. Все знакомые чат-

модели — из этой ветки, и в книге мы строим именно её: «decoder-only» трансформер.

1.3 Три этапа создания модели (план книги)

1. **Данные и архитектура (главы 2–4).** Готовим текст (токены, эмбеддинги) и собираем саму нейросеть: механизм внимания + окружающие его слои.
2. **Предобучение (глава 5).** Прогоняем через модель большой корпус текста, обучая предсказывать следующий токен. Результат — **базовая модель**. У гигантов это стоит миллионы долларов; мы потренируемся на крошечном тексте, а потом загрузим в нашу архитектуру настоящие веса GPT-2, опубликованные OpenAI.
3. **Дообучение (главы 6–7).** Специализируем базовую модель: в главе 6 — в классификатор (спам/не спам), в главе 7 — в помощника, следующего инструкциям. Это тот самый шаг, который отделяет «автодополнялку» от ChatGPT.

Модель книги — **GPT-2** (OpenAI, 2019): она открыта, хорошо изучена и достаточно мала, чтобы обучаться на обычном ноутбуке. Архитектурно современные модели (Llama, Qwen из первого пособия) — её прямые потомки с точечными улучшениями.

> **Ключевая мысль главы 1.** LLM — трансформер-декодер, обученный предсказывать следующий токен; из этой простой задачи при масштабе вырастают сложные умения. Создание модели — три этапа: архитектура и данные → предобучение (дорого, результат — базовая модель) → дообучение (дёшево, результат — специалист или помощник).

Вопросы к главе 1

1. Почему задача «предскажи следующее слово» заставляет модель выучить грамматику, факты и логику, хотя её этому не учили специально?
2. Что такое трансформер и какой механизм является его сердцем?
3. Чем ветка GPT (декодеры) отличается от ветки BERT (энкодеры) по назначению?
4. Назовите три этапа создания LLM. Какой из них самый дорогой и что является его результатом?
5. Почему для учебной книги выбрана именно GPT-2, а не современная гигантская модель?

Глава 2. Данные: от сырого текста до входа модели

Нейросеть ест только числа. Эта глава — конвейер превращения текста в числа, пригодные для обучения: токенизация → номера → эмбеддинги → обучающие пары. Часть вы знаете из первого пособия, но теперь мы строим токенизатор сами и смотрим глубже.

2.1 Самодельный токенизатор и его провал

Книга начинает с наивного токенизатора: режем текст по пробелам и знакам препинания, собираем **словарь** — список всех уникальных слов рассказа (учебный текст — рассказ «The Verdict», ~5 тысяч токенов), каждому слову даём номер. Класс с двумя методами: `encode` (текст → номера) и `decode` (номера → текст).

Такой токенизатор мгновенно ломается: встретив слово, которого не было в словаре (хоть «Hello»), он не знает, какой номер выдать. Первый костыль — специальный токен `<|unk|>` («неизвестное слово»), но это потеря информации: все незнакомые слова сливаются в одно пятно.

2.2 Настоящее решение: BPE

> **BPE (Byte Pair Encoding)** — алгоритм токенизации GPT-моделей. Словарь строится не из слов, а из **частых кусочков**: сначала отдельные символы, затем алгоритм многократно склеивает самые частые пары соседних кусочков в новые токены. Частые слова становятся одним токеном, редкие собираются из нескольких: «unfamiliarword» → [«unfam», «iliar», «word»]. Незнакомых слов не существует в принципе: в худшем случае слово разберётся на отдельные символы.

У GPT-2 словарь BPE — **50 257 токенов**. Из специальных токенов GPT-2 обходится одним-единственным `<|endoftext|>`: он и разделитель документов, и заполнитель. (Сравните со 151 936 токенами Qwen3 из первого пособия — идея та же, словарь больше.)

> **Аналогия.** BPE — как конструктор LEGO: для частых построек есть готовые крупные блоки («дом»), всё редкое собирается из мелких деталей. Ни одна постройка не невозможна — вопрос лишь в количестве деталей.

2.3 Обучающие пары: скользящее окно

Как из сплошного текста сделать обучающие примеры? Гениально просто: **цель (target) — это вход (input), сдвинутый на один токен вправо**.

```
текст:  «и в путь дорога зовёт нас»
вход:   [и, в, путь, дорога]
цель:   [в, путь, дорога, зовёт]
```

На каждой позиции модель должна по началу предсказать следующий токен — одна строка текста даёт сразу несколько задачек. По тексту двигается **скользящее окно** фиксированной длины (длина = контекст модели), нарезая его на такие пары. Никакой ручной разметки: любой текст сам себе учебник. Именно поэтому предобучение называют **самообучением** (self-supervised) — метки создаются из самих данных.

2.4 Эмбеддинги: номер токена → вектор смысла

Номер токена — просто индекс, никакого смысла в нём нет (токен 5131 не «ближе» к 5132, чем к 100). Смысл появляется на следующем шаге.

> **Слой эмбеддингов (embedding layer)** — таблица размером «словарь × размерность», в которой каждому токену соответствует строка — его вектор. Превращение токена в вектор — буквально **взятие строки по индексу** (lookup). Главное: эта таблица — **обучаемые веса**. В начале там случайные числа, и в процессе обучения модель сама расставляет слова в пространстве так, чтобы похожие по употреблению оказались рядом.

```
# по сути embedding - это просто:
emb_table = Matrix(vocab_size=50257, dim=768)   # обучаемая
vector = emb_table[token_id]                     # взять строку
```

2.5 Позиционные эмбеддинги: где стоит слово

Проблема: lookup выдаёт для токена один и тот же вектор, где бы тот ни стоял. Но «собака укусила человека» и «человек укусил собаку» — разные тексты! Внимание (глава 3) само по себе не различает порядок слов — ему нужно подсказать позицию.

> **Позиционный эмбеддинг** — вторая обучаемая таблица: «позиция → вектор» (вектор позиции 0, позиции 1, … до длины контекста). Итоговый вход модели = **вектор токена + вектор его позиции** (поэлементная сумма). Теперь одно и то же слово на разных местах выглядит для модели по-разному.

Ключевая мысль главы 2. Конвейер данных: текст → BPE-токены (словарь из частых кусочков, незнакомых слов не бывает) → номера → вектор токена из обучаемой таблицы + вектор позиции. Обучающие пары делаются из любого текста бесплатно: цель = вход со сдвигом на 1 (самообучение).

Вопросы к главе 2

1. Почему токенизатор «по словам» ломается на новых текстах и почему токен <|unk|> — плохое решение?
2. Как BPE строит словарь и почему для него не существует незнакомых слов?
3. Как из сплошного текста получаются пары «вход → цель»? Почему это называют самообучением?
4. Что такое слой эмбеддингов, как он превращает номер в вектор и почему это обучаемая часть модели?
5. Зачем к вектору токена прибавляют вектор позиции? Что сломалось бы без этого?

Глава 3. Внимание: двигатель LLM

Самая важная глава книги. Механизм внимания — то, что отличает трансформер от всего, что было до него. Разберём медленно, в два прохода: сначала упрощённую версию (чтобы понять суть), потом настоящую (как в GPT). Вам будет легче, чем другим читателям: с Q, K и V вы уже встречались, когда разбирали KV-кэш.

3.1 Зачем вообще нужно внимание

Возьмём предложение: «Робот взял книгу со стола, потому что **она** была нужна для урока». Чтобы понять слово «она», надо связать его с «книгой» — через четыре слова назад. Смысл слова зависит от далёкого контекста.

До трансформеров текст обрабатывали рекуррентные сети (RNN): читали слово за словом, таща с собой один вектор-«память» обо всём прочитанном. На длинных текстах это узкое горлышко: к концу предложения начало уже «забыто». Идея внимания: **не сжимать прошлое в одну память, а дать каждому слову напрямую посмотреть на все остальные слова и взять из них нужное**.

3.2 Упрощённое внимание: суть на пальцах

Вход: векторы токенов $x^{(1)}...x^{(T)}$ (из главы 2). Выход: для каждого токена — новый вектор, **контекстный**, который обогащён информацией от соседей. Для токена №2 это делается в три шага:

1. **Похожесть.** Считаем скалярное произведение вектора $x^{(2)}$ с каждым вектором последовательности (включая себя). Получаем T чисел — «оценки внимания»: насколько каждый токен релевантен токену №2. (Вот зачем в главе 0 было скалярное произведение!)
2. **Нормировка.** Прогоняем оценки через softmax — функцию, превращающую любой набор чисел в положительные веса с суммой 1. Получились «веса внимания»: скажем, [0.1, 0.5, 0.1, 0.3] — токен №2 на 50% «смотрит» на себя, на 30% — на токен №4.
3. **Смесь.** Контекстный вектор $z^{(2)}$ = взвешенная сумма всех входных векторов с этими весами. Это «новая версия» токена №2, в которую подмешана информация от релевантных ему слов.

Аналогия. Слово на совещании: прежде чем высказаться (обновить свой смысл), оно опрашивает всех участников, оценивает, чьё мнение релевантно (похожесть), раздаёт доли влияния (веса в сумме 100%) и формирует позицию как взвешенное среднее услышанного.

3.3 Настоящее внимание: Q, K, V и при чём тут «поиск»

В упрощённой версии токен ищет «похожих на себя». Это слишком грубо: слову «она» нужно найти не похожее слово, а существительное женского рода — критерий поиска не совпадает с собственным содержанием. Решение: перед сравнением каждый токен проецируется **тремя обучаемыми матрицами** W_q, W_k, W_v в три разных вектора:

- **Q (query)** = x·W_q — «что я ищу»;
- **K (key)** = x·W_k — «по какому признаку меня можно найти»;
- **V (value)** = x·W_v — «что я отдам, если меня выбрали».

```
scores  = Q @ K.T / sqrt(d_k)   # похожесть запросов и ключей
weights = softmax(scores)        # по строкам, сумма = 1
Z       = weights @ V            # смесь значений
```

Схема та же, что в 3.2 (похожесть → softmax → смесь), но сравниваются Q с K, а смешиваются V. Деление на √d_k (отсюда название «scaled dot-product attention») — техническая стабилизация: без него при больших размерностях скалярные произведения огромны, softmax «зашкаливает» и обучение буксует. **Главное же отличие — матрицы W_q, W_k, W_v обучаемы**: модель сама, градиентным спуском, выучивает, что искать, по чему искать и что передавать. Внимание — не захардкоженное правило, а выученный навык.

Знакомо? В первом пособии вы узнали K и V как «карточки», которые сохраняет KV-кэш. Теперь вы знаете, откуда они берутся: это проекции токена через обучаемые матрицы.

3.4 Каузальная маска: не подглядывать в будущее

GPT предсказывает следующий токен. Если при обработке токена №3 внимание сможет заглянуть в токен №4, обучение превратится в фарс: ответ виден в условии. Поэтому в GPT используют **каузальное** (причинное) внимание: каждый токен смотрит только на себя и налево.

Реализация элегантна: в матрице оценок внимания все ячейки «выше диагонали» (будущее) заменяются на −∞ **до** softmax. Softmax превращает −∞ в вес ровно 0, и строки по-прежнему суммируются в 1. Почему не занулить веса после softmax? Тогда суммы строк перестанут равняться 1 и придётся нормировать заново — лишняя работа.

Дополнительно при обучении применяется **dropout** — случайное обнуление части весов внимания (например, 10%), чтобы модель не привыкала полагаться на одни и те же связи (защита от переобучения). На инференсе dropout выключен.

3.5 Многоголовое внимание

Multi-head attention — несколько механизмов внимания («голов»), работающих параллельно, каждая со своими матрицами W_q, W_k, W_v. Результаты голов склеиваются. У GPT-2 small — 12 голов в каждом слое.

Зачем: одна голова — один «критерий поиска» на токен, а связей в языке много одновременно. Одна голова может выучиться связывать местоимения с существительными, другая — глаголы с подлежащими, третья — следить за запятыми. Головы — это параллельные «специалисты», смотрящие на текст каждая со своей точки зрения. (Технически их реализуют эффективно: одна большая матрица, результат которой режется на головы, — но математика эквивалентна нескольким независимым вниманиям.)

Ключевая мысль главы 3. Внимание обновляет смысл каждого токена смесью других токенов: похожесть (скалярное произведение Q и K) → softmax в веса → взвешенная сумма V. Матрицы Q/K/V обучаемы — модель сама учится, что и как искать. Каузальная маска ($-\infty$ до softmax) запрещает смотреть в будущее, а 12 параллельных голов ловят разные типы связей одновременно.

Вопросы к главе 3

1. Какую проблему старых (рекуррентных) сетей решает внимание? В чём была их «узость»?
2. Опишите три шага вычисления контекстного вектора в упрощённом внимании.
3. Зачем токен проецируют в три разных вектора Q, K, V вместо того чтобы сравнивать «сырые» векторы токенов? Что означает каждый из трёх?
4. Как устроена каузальная маска и почему $-\infty$ вставляют до softmax, а не зануляют веса после?
5. Зачем нужно несколько голов внимания, если можно сделать одну побольше?

Глава 4. Собираем GPT: архитектура целиком

Внимание — двигатель, но автомобиль состоит не только из двигателя. В этой главе вокруг внимания собирается весь «кузов», и получается настоящая GPT-2. Деталей четыре, затем сборка.

4.1 Паспорт модели: конфигурация GPT-2 small

```
vocab_size     = 50257   # словарь BPE (глава 2)
context_length = 1024    # максимум токенов за раз
emb_dim        = 768     # размерность векторов токенов
n_heads        = 12      # голов внимания в каждом слое
n_layers       = 12      # трансформерных блоков
drop_rate      = 0.1     # dropout
Итого: 124 миллиона параметров
```

4.2 Деталь 1: LayerNorm — стабилизатор

> **Layer Normalization** — слой, который приводит числа в векторе к стандартному виду: среднее 0, разброс 1 (плюс два обучаемых параметра масштаба и сдвига). Ставится перед вниманием, перед feed-forward и перед выходным слоем.

Зачем: в глубокой сети (12 блоков подряд) значения от слоя к слою норовят разрастись или затухнуть, и обучение разваливается. LayerNorm после каждого «этажа» возвращает числа в рабочий диапазон — как стабилизатор напряжения между приборами.

4.3 Деталь 2: Feed-Forward с GELU — «обдумывание»

> **Feed-forward (FFN)** — маленькая двухслойная сеть внутри каждого блока: расширить вектор в 4 раза (768 → 3072), применить нелинейность **GELU** (гладкий аналог ReLU: отрицательные значения не рубятся в ноль, а мягко прижимаются), сжать обратно (3072 → 768). Применяется к каждому токену независимо.

Разделение труда в блоке: **внимание — это обмен информацией между токенами** («посмотри на соседей»), **FFN — индивидуальная переработка собранного каждым токеном** («обдумай, что услышал»). Расширение в 4 раза

даёт «рабочий стол» побольше, чтобы развернуть промежуточные вычисления, потом результат компактно упаковывается обратно.

4.4 Деталь 3: остаточные связи — лифт для градиента

> **Остаточная связь (shortcut / residual connection)** — выход слоя **прибавляется** к его входу: out = x + layer(x). То есть слой учит не «новое представление с нуля», а **поправку** к уже имеющемуся.

Зачем: при обучении градиент течёт от потери назад сквозь все слои, затухая на каждом (проблема «исчезающего градиента» — ранние слои почти не учатся). Остаточная связь — обходной путь: градиент может протечь напрямую, минуя слой. В книге это показано экспериментом: без остаточных связей градиенты в первых слоях глубокой сети практически нулевые, с ними — живые. Именно остаточные связи позволяют строить сети в десятки и сотни слоёв.

4.5 Сборка: трансформерный блок и вся модель

```
# Один трансформерный блок (этаж):
x = x + MultiHeadAttention(LayerNorm(x))   # обмен между токенами
x = x + FeedForward(LayerNorm(x))          # переработка каждым токеном

# Вся GPT-2:
векторы = ТокенЭмбеддинг(токены) + ПозиционныйЭмбеддинг(позиции)   # гл. 2
x = 12 × ТрансформерныйБлок(векторы)                                # этажи
логиты = ФинальныйLayerNorm(x) @ ВыходнаяМатрица                    # 768 → 50257
```

Вся громада GPT — это один и тот же блок, повторённый 12 раз (у GPT-2 XL — 48, у гигантов — под сотню). Модель «большая» не кодом (несколько сотен строк!), а числом параметров. Выходная матрица проецирует 768-мерный вектор каждого токена обратно в 50 257 логитов — оценок для каждого токена словаря. Круг замкнулся: это те самые логиты из главы 0.

Любопытная деталь: честный подсчёт параметров даёт 163 млн, а не 124. В оригинальной GPT-2 входная таблица эмбеддингов и выходная матрица — это **одна и та же** матрица (weight tying, 50257×768 ≈ 39 млн параметров экономии). В книге их держат раздельно — так обучение идёт лучше.

Собранная модель уже умеет генерировать текст (жадным циклом из первого пособия) — но веса случайны, и она выдаёт бессмыслицу. Учить будем в главе 5.

> **Ключевая мысль главы 4.** GPT = эмбеддинги + N одинаковых блоков + проекция в логиты. Блок = внимание (токены обмениваются) + FFN (каждый обдумывает), всё обёрнуто в LayerNorm (стабилизация чисел) и остаточные связи (x + слой(x) — чтобы градиент доходил до ранних слоёв). 124 млн параметров, ~12 этажей, несколько сотен строк кода.

Вопросы к главе 4

1. Какое разделение труда между вниманием и feed-forward-сетью внутри трансформерного блока?
2. Зачем нужен LayerNorm и где он стоит в блоке?
3. Как записывается остаточная связь и какую проблему глубоких сетей она решает?
4. Из чего состоит один трансформерный блок и как из блоков собирается вся модель? Что делает самый последний слой модели?
5. Модель называется 124-миллионной, но честный подсчёт даёт 163 млн. Откуда разница?

Глава 5. Предобучение: модель учится писать

Архитектура готова, веса случайны. Пора учить. В этой главе: чем измерять «качество текста» (loss и perplexity), сам цикл обучения, наглядное переобучение, управление случайностью при генерации и загрузка настоящих весов GPT-2 от OpenAI.

5.1 Функция потерь: кросс-энтропия

Вспомним: обучающая пара — это вход и цель (вход со сдвигом 1). Модель выдаёт для каждой позиции распределение вероятностей по словарю. Насколько она ошиблась?

> **Кросс-энтропия (cross-entropy)** — функция потерь для предсказания «одного из многих»: смотрим, какую вероятность модель присвоила **правильному** следующему токену, берём логарифм, усредняем по всем позициям, ставим минус. Присвоила правильному токену вероятность 1 → потеря 0 (идеал); присвоила крошечную → потеря большая. Минимизировать кросс-энтропию = заставлять модель давать правильным токенам максимальную вероятность.

Это ровно те лог-вероятности из первого пособия (глава 5 про уверенность и глава 8 про дистилляцию — там мы уже выяснили, что кросс-энтропия и средний logprob — одно и то же с точностью до знака). Круг замкнулся окончательно: **предобучение LLM, дистилляция и оценка уверенности построены на одной и той же формуле**.

> **Перплексия (perplexity)** = exp(кросс-энтропия) — та же метрика в удобных единицах: «из скольких токенов модель как будто выбирает наугад». Перплексия 48 725 (наша необученная модель) — растерянность на уровне «любой из почти всего словаря»; перплексия хорошей модели — единицы или десятки: «колеблюсь между десятком вариантов».

5.2 Цикл обучения и честная цена вопроса

Сам цикл обучения — классика из главы 0, без всякой магии:

```
for epoch in epochs:
    for inputs, targets in train_loader:      # пары из главы 2
        loss = cross_entropy(model(inputs), targets)
        loss.backward()                        # градиенты
```

```
        optimizer.step()                        # шаг по весам (AdamW)
        optimizer.zero_grad()
```

Книга обучает модель на одном коротком рассказе (~5 тысяч токенов) — несколько минут на ноутбуке. Для сравнения настоящая цена: Llama 2 7B обучалась на 2 триллионах токенов, ~184 тысячи GPU-часов — порядка $690 000 только за аренду железа (а у передовых моделей — на порядки больше). Поэтому стратегия книги: механику показать на игрушке, веса взять готовые.

5.3 Переобучение вживую

На таком крошечном датасете модель быстро начинает... **цитировать рассказ дословно**. Потеря на обучающем тексте падает к нулю, на валидационном — растёт: классическое переобучение (глава 0) в самом наглядном виде — модель не «научилась писать», а вызубрила единственную книжку. Для учебных целей это даже полезно: видно, что модель способна учиться, и видно, как выглядит зубрёжка на графиках. Лекарство — больше данных (настоящие модели видят каждый кусок текста около одного раза — зубрить просто некогда).

5.4 Управление случайностью: температура и top-k

Знакомые по первому пособию ручки, теперь — их реализация:

- Жадный выбор: `argmax(логиты)` — всегда одно и то же.
- Сэмплирование: `multinomial(softmax(логиты))` — розыгрыш по вероятностям.
- **Температура**: `softmax(логиты / T)` — T<1 заостряет распределение, T>1 сглаживает. В книге показан живой пример риска: при высокой температуре модель в 3,2% случаев продолжала «every effort moves you» словом «pizza».
- **Top-k**: оставить k самых вероятных токенов, остальным выставить $-\infty$ (знакомый трюк из каузальной маски!), затем softmax и розыгрыш. Родной брат top-p из первого пособия: top-k фиксирует количество кандидатов, top-p — их суммарную вероятность.

5.5 Загрузка весов OpenAI

Финал главы: скачиваем опубликованные OpenAI веса GPT-2 (124M) и аккуратно раскладываем их по матрицам нашей самодельной архитектуры. Если архитектура воспроизведена точно — чужие веса «встанут» и модель заговорит связно. Так и происходит — и это сильная проверка: малейшая ошибка в архитектуре дала бы бессмыслицу. Доступны и старшие братья: 355M (medium), 774M (large), 1558M (XL) — та же схема, больше этажей и размерности.

Ключевая мысль главы 5. Предобучение — это минимизация кросс-энтропии (= максимизация вероятности правильного следующего токена) градиентным спуском по парам «вход → вход со сдвигом». Перплексия = exp(потери) — «из скольких вариантов гадаю». На крошечных данных модель зубрит (переобучение); настоящее предобучение стоит сотни тысяч долларов, поэтому мы загружаем готовые веса GPT-2 в свою архитектуру — и она оживает.

Вопросы к главе 5

1. Что именно измеряет кросс-энтропия при обучении LLM и каким было бы её значение у идеальной модели?
2. Что такое перплексия и как интерпретировать значение 48 725 у необученной модели?
3. Как выглядело переобучение при обучении на одном рассказе? По каким признакам его распознали?
4. Чем top-k отличается от top-p (вспомните первое пособие) и каким техническим трюком top-k отсекает лишние токены?
5. Почему успешная загрузка чужих весов GPT-2 — сильное доказательство, что архитектура из главы 4 реализована правильно?

Глава 6. Дообучение №1: LLM как классификатор

База есть. Теперь — первый вид специализации: превратить генератор текста в **классификатор**, отвечающий ровно на один вопрос. Учебная задача: спам-фильтр для СМС (датасет ~1,5 тыс. сообщений: спам / не спам).

6.1 Два вида дообучения

> **Классификационное дообучение** — модель учат выдавать одну из фиксированных меток (спам/не спам). Узкий специалист: ничего другого делать не сможет, зато задачу решает надёжно и дёшево. **Инструкционное дообучение** (глава 7) — модель учат выполнять произвольные текстовые команды. Универсал, но его сложнее сделать хорошо. Специалиста создать гораздо проще, чем универсала.

6.2 Хирургия: замена головы

Трюк главы — минимальная операция над готовой моделью:

1. Берём предобученную GPT-2 (124M) с весами OpenAI.
2. **Отрезаем выходной слой** (проекция 768 → 50 257 логитов словаря) — он нам не нужен, мы не генерируем текст.
3. **Пришиваем новый**: крошечная матрица 768 → 2 (два логита: «не спам» / «спам»).
4. **Замораживаем** почти всю модель (веса не обновляются) и обучаем только новую голову + последний трансформерный блок + финальный LayerNorm.

Почему это работает: за время предобучения модель уже выучила язык — её внутренние векторы насыщены полезными признаками. Для классификации достаточно научиться эти признаки считывать (новая голова), слегка подстроив верхний этаж. Заморозка делает обучение быстрым (минуты), защищает выученное от разрушения и требует в разы меньше данных.

Аналогия. Опытного редактора переучивают на модератора: не учат заново читать (нижние слои заморожены), а дают новую инструкцию оценки (новая голова) и короткую стажировку (дообучение верхнего слоя).

6.3 Деталь: почему смотрим на последний токен

Модель выдаёт вектор для **каждого** токена сообщения — а метка нужна одна на всё сообщение. Чей вектор брать? **Последнего токена**. Вспомните каузальную маску (глава 3): каждый токен видит только себя и то, что левее. Единственный токен, «прочитавший» всё сообщение, — последний. Его вектор и есть резюме всего текста, по нему и классифицируем.

Обучение — та же кросс-энтропия (теперь между 2 классами вместо 50 257 токенов — какая ирония: предсказание следующего токена всегда и было классификацией на 50 257 классов). Точность (процент угаданных меток) нельзя оптимизировать напрямую — она не дифференцируема (ступенчатая: чуть сдвинул вес — точность не меняется, градиент нулевой), поэтому кросс-энтропия служит её гладкой заменой.

Результат: ~96% точности на тестовой выборке после 5 эпох (минуты обучения на ноутбуке). Проверка честности — три набора данных: обучающий (на нём учимся), валидационный (подбираем настройки), тестовый (трогаем один раз в конце — честная оценка).

> **Ключевая мысль главы 6.** Готовую LLM можно дёшево превратить в классификатор: заменить словарную голову на голову из N классов, заморозить почти всё, дообучить верхушку. Классифицируем по вектору последнего токена — единственного, который видел весь текст. Предобученные признаки + крошечное дообучение = 96% точности за минуты.

Вопросы к главе 6

1. Чем классификационное дообучение отличается от инструкционного и почему специалиста сделать проще?
2. Опишите «хирургию» подготовки модели: что отрезали, что пришили, что заморозили?
3. Почему для классификации берут вектор именно последнего токена? (Подсказка: глава 3.)
4. Почему нельзя обучаться напрямую на точность и что используют вместо неё?
5. Зачем три набора данных — обучающий, валидационный, тестовый? Чем принципиален тестовый?

Глава 7. Дообучение №2: учим следовать инструкциям

Финальная глава — рождение помощника. Базовая модель на просьбу «исправь грамматику в предложении» отвечает... повторением просьбы или списком похожих заданий (вы предсказали это ещё в главе 0 первого пособия!). Учим её вести себя как ассистент.

7.1 Данные: пары «инструкция → ответ»

Датасет — 1100 рукотворных примеров вида:

```
{"instruction": "Перепиши предложение в пассивном залоге",
 "input":       "The chef cooks the meal every day.",
 "output":      "The meal is cooked every day by the chef."}
```

Перед подачей в модель каждый пример упаковывается в единый текстовый шаблон (книга использует формат Alpaca): служебные заголовки «### Instruction:», «### Input:», «### Response:» и после последнего — ожидаемый ответ. Обучение — то же предсказание следующего токена на этих текстах (это и называется supervised fine-tuning, SFT). Модель выучивает паттерн: после «### Response:» надо писать ответ на инструкцию, а не продолжать болтовню.

Знакомо из первого пособия? Это ровно тот механизм, которым в главе 8 делали дистилляцию: там SFT шёл на ответах модели-учителя, здесь — на ответах людей. Механика одна.

7.2 Техника батчей: паддинг и маскирование потерь

Примеры разной длины, а в батч (пачку для параллельного обучения) нужны одинаковые. Решение в два приёма:

- **Паддинг:** короткие примеры в батче добиваются токеном `<|endoftext|>` до длины самого длинного (причём в этой главе — до длинного в своём батче, а не во всём датасете, — меньше лишней набивки).
- **Маскирование потерь:** в целях (targets) все хвостовые паддинг-токены заменяются на специальное значение **−100** — PyTorch-овская кросс-энтропия по договорённости **игнорирует** позиции с этой меткой. Смысл: не учить модель генерировать набивку. Первый `<|endoftext|>` после ответа при этом оставляют как цель — модель должна выучить, где остановиться. (Распространённая практика — маскировать и токены самой инструкции, чтобы учиться только на ответах; в книге это вынесено в упражнение.)

7.3 Обучение и главный сюрприз: как оценивать?

Обучается модель покрупнее — GPT-2 medium, 355M (малышка 124M для связных ответов слабовата). Пара эпох — и модель реально следует инструкциям: переводит в пассивный залог, отвечает на вопросы. Но тут всплывает проблема, которой не было в главе 6: **как измерить качество свободного текстового ответа?** Точность не посчитаешь: правильных формулировок бесконечно много, а ответ «cumulus cloud» вместо «cumulonimbus» — не 0 и не 1, а «почти».

Варианты из практики: экзамены с вариантами ответов (MMLU), сравнение людьми (арены), и — подход книги — **LLM-судья**: более сильная модель (локальная Llama 3 8B через Ollama) получает инструкцию, эталонный ответ и ответ нашей модели и выставляет балл 0–100. Это автоматизируемо и неплохо коррелирует с человеческими оценками.

Результат: наша дообученная GPT-2 355M набирает в среднем ~50 баллов (для ориентира тем же судьёй: Llama 3 8B base — 58,5; Llama 3 8B instruct — 82,7). Скромно, но честно: модель 2019 года на 1100 примерах против моделей на три порядка дороже.

Сравните с первым пособием: там оценка была лёгкой (математика, верификатор, 0/1). Свободный текст оценивать принципиально труднее — вот почему там мы начинали именно с математики, а здесь приходится звать LLM-судью.

7.4 Что дальше: мостик к первому пособию

Книга заканчивается там, где начиналось первое пособие. Полный конвейер современного помощника:

```
предобучение → базовая модель            (эта книга, гл. 2-5)
SFT на инструкциях → помощник            (эта книга, гл. 7)
[опционально] преференс-тюнинг (DPO/RLHF) → вежливый помощник
[опционально] RLVR/GRPO, дистилляция → рассуждающая модель   (первое пособие)
```

Теперь у вас есть обе половины картины: как модель устроена и обучена — и как её учат рассуждать.

> **Ключевая мысль главы 7.** Инструкционное дообучение (SFT) — предсказание следующего токена на текстах-шаблонах «инструкция → ответ»; после него модель ведёт себя как помощник. Технические детали: паддинг батчей токеном конца текста и метка −100, выключающая набивку из функции потерь. Оценка свободных ответов — отдельная трудная задача; практичное решение — LLM-судья.

Вопросы к главе 7

1. Почему базовая модель не выполняет инструкции и как SFT это чинит, оставаясь всё тем же «предсказанием следующего токена»?
2. Зачем нужен паддинг и зачем паддинг-токены в целях заменяют на -100?
3. Почему первый токен конца текста после ответа НЕ маскируют в целях?
4. Почему оценка инструкционной модели сложнее, чем классификатора из главы 6? Какие подходы существуют и какой выбран в книге?
5. Соберите полный конвейер создания современного чат-помощника из этапов обеих книг (этой и первого пособия). Что на каком этапе появляется?

Заключение

Маршрут одним абзацем. Текст нарезается BPE-токенизатором на кусочки, каждый получает обучаемый вектор плюс вектор позиции (гл. 2). Сердце модели — внимание: Q/K/V-проекции, похожесть через скалярное произведение, softmax, взвешенная смесь; каузальная маска запрещает подглядывать в будущее, 12 голов ловят разные связи (гл. 3). Блок = внимание + feed-forward в обёртке LayerNorm и остаточных связей; 12 блоков + выходная проекция = GPT-2 (гл. 4). Предобучение минимизирует кросс-энтропию следующего токена; перплексия мерит растерянность; настоящие веса грузим от OpenAI (гл. 5). Дообучение: замена головы + заморозка = классификатор по последнему токену (гл. 6); SFT на шаблонах «инструкция → ответ» с маскированием −100 = помощник, оцениваемый LLM-судьёй (гл. 7).

Куда дальше:

- Репозиторий книги: **github.com/rasbt/LLMs-from-scratch** — весь код по главам + россыпь бонусов (KV-кэш, Llama 3.2 с нуля, DPO, LoRA, MoE).
- Первое пособие этой серии — рассуждающие модели (если ещё не закончили — самое время вернуться с новым пониманием «начинки»).
- Блог автора: magazine.sebastianraschka.com.

Порядок работы прежний: ответы на вопросы главами присылайте в чат («Глава N: 1) … 5) …») — проверяю, объясняю, двигаемся. Удачи!